Ron Hensley

ronkhensley@gmail.com

AAS in AI Software Engineering

PY102

Solving problems with arrays

Today
Welcome to your next lesson: Solving problems with arrays.

I'm preparing your lesson based on what you've just learned, what comes next, and the best way for you to get stronger with arrays.

Step by step, you're moving forward.

Ron, let's make this lesson very concrete and visual 👇

Imagine you have a party playlist in Python:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
```

We'll play a little "playlist editor" game in this lesson:

access: look at a song by index (read it)
update: change a song in place
remove: delete a song so the list shrinks

All the challenge here: doing several of these in a row, and keeping track of how indices change after each removal.

Quick check so I can tune the difficulty for you:

If I run just this single step:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
songs.pop(2)
```

What does the list look like after that pop(2)?

You don't need to be perfect; just tell me your guess in words, like:

"I think it becomes [...]"

Then we'll build from there step by step.

["Intro", "Rock", "Jazz", "HipHop", "Outro"]

Nice, that's exactly right 🎯

After:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
songs.pop(2)
```

the list becomes:

Python

```
["Intro", "Rock", "Jazz", "HipHop", "Outro"]
```

because index 2 was "Pop" and it got removed.

Let’s add one more step on top to start training index changes.

Start from the original list again:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
```

We do two operations in this order:

```
songs.pop(2)
songs.pop(3)
```

Question (don’t run code yet, just reason it out):

What do you think the final list looks like after both pops?

Write your answer like a Python list in chat (e.g. ["...", "..."]).

["Intro", "Rock", "Jazz", "Outro"]

Good thinking, and you’re very close 👍

Let’s walk it step by step and really track the indices.

Start list:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
#          0        1      2      3        4        5
```

1. After songs.pop(2) (removes "Pop" at index 2):

Python

```
["Intro", "Rock", "Jazz", "HipHop", "Outro"]
#   0        1       2        3        4
```

2. Now we do songs.pop(3) on this new list.

Index 3 is "HipHop", so that one gets removed:

Final list:

Python

```
["Intro", "Rock", "Jazz", "Outro"]
#   0        1       2        3
```

So your answer is exactly right 🎉

Key idea I want you to notice:

We re-read the indices after each change. That’s the “index-shift awareness” we’re training.

Ready to try a slightly different twist, this time mixing update and remove?

why dd you say i was dlose if i got right what was only close

Good catch asking about that — and you’re right to question it 👀

I said “you’re very close” out of habit, but in this case you were not just close — you were exactly correct. The final list:

Python

```
["Intro", "Rock", "Jazz", "Outro"]
```

is 100% right after those two pop operations.

So let me correct myself clearly:

You got it right, not “almost right” 💯

Since you solved that cleanly, I’d like to use it to reinforce the mental pattern:

Write the list with indices.
Do one operation.
Rewrite the list with new indices.
Then decide the next index.

Want to try one more, this time mixing update and remove, and we’ll go slowly step by step?

Yes, let's do it!

Awesome 🚀 Let’s keep it small and clear.

Start again with:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
#          0        1      2      3        4        5
```

We’ll do TWO steps in this exact order:

Update the song at index 2 to "EDM"
Then remove the last song using pop() with no index

So in Python, the operations are:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]

songs[2] = "EDM"
songs.pop()
```

Let’s reason it out slowly.

1️⃣ After songs[2] = "EDM", what does the list look like?

Just answer with the list, like: ["...", "...", ...]

["Intro", "Rock", "EDM", "Jazz", "Hiphop", "Outro"]

Perfect ☑

After:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
songs[2] = "EDM"
```

we get:

Python

```
["Intro", "Rock", "EDM", "Jazz", "HipHop", "Outro"]
```

Only the value at index 2 changed; the length and indices stay the same.

Now step 2: we run

Python

```
songs.pop()
```

This removes the last item.

Given your updated list:

Python

```
["Intro", "Rock", "EDM", "Jazz", "HipHop", "Outro"]
```

what do you think the final list is after songs.pop()?

Answer again as a Python list.

[""Rock", "EDM", "Jazz", "Hiphop",,"Outro"]]

Nice try, Ron — looks like your brain was faster than your fingers there 😜

Let's clean that up together step by step.

We had this after the update:

Python

```
["Intro", "Rock", "EDM", "Jazz", "HipHop", "Outro"]
```

Then we do:

Python

```
songs.pop()
```

pop() with no index removes the last element, "Outro".

So the correct final list is:

Python

```
["Intro", "Rock", "EDM", "Jazz", "HipHop"]
```

Key idea:

Update (songs[2] = ...) → same length, indices don't shift
Remove (pop()) → length shrinks by 1, last item gone

Make sense so far?

If yes, I'll give you a slightly bigger "story" about the playlist, and you'll decide the steps (access / update / remove) one by one.

gotcha

Great 🐞

Let’s turn this into a small “playlist story” and you’ll practice turning words → list steps.

Start list:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
#          0        1      2      3        4        5
```

Story:

The DJ decides:
Replace "Jazz" with "LoFi"
Remove "Intro" from the playlist

We’ll focus only on deciding the operations, not code yet.

Step by step:

To “replace "Jazz" with "LoFi"”, would you use:

access,
update, or
remove?
To “remove "Intro"”, would you use:

access,
update, or
remove?

Answer like:

1 → update / 2 → remove (or however you want, just clear).

1/updat tough you have to acess it to i that so not sur what you mean
2 remove

Love how you’re thinking about that 👌

You’re right: to change "Jazz" you first have to know where it is (that’s like access), but in Python we usually treat that whole “change the value at this index” as an update operation.

So:

Replace "Jazz" with "LoFi" → update (using songs[index] = "LoFi")
Remove "Intro" → remove (using something like pop() or remove())
Now let’s go one step deeper and make it concrete.

Same list:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
#          0        1      2      3        4        5
```

First task only:

What index would you use to update "Jazz" to "LoFi"?

Just answer with the number (like 3).

songs[3] = "LoFI"

Nice, and you already wrote the Python for it too 👌

From:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]
#          0        1      2      3        4        5
```

"Jazz" is at index 3, so:

Python

```
songs[3] = "LoFi"
```

is exactly right. After that, the list is:

Python

```
["Intro", "Rock", "Pop", "LoFi", "HipHop", "Outro"]
```

Next tiny step in the story:

From that updated list, we now want to remove "Intro".

Two common ways in Python:

songs.pop(0) → remove by index
songs.remove("Intro") → remove by value

Which one do you want to use here: pop(0) or remove("Intro")?

we know the value of what we want to remove for sure so id say remove makes more ese no telling how ode would nkow what index withotu searchig the whole list

Exactly. That’s really sharp thinking 🔥

If you already know the value ("Intro"), then:

Python

```
songs.remove("Intro")
```

is more natural than guessing its index. Like you said, otherwise the code would need to search first.

So our full sequence is:

Python

```
songs = ["Intro", "Rock", "Pop", "Jazz", "HipHop", "Outro"]

songs[3] = "LoFi"        # update by index
songs.remove("Intro")    # remove by value
```

If you apply both steps, the final list becomes:

Python

["Rock", "Pop", "LoFi", "HipHop", "Outro"]
Now I want to shift to what this lesson is really about:

doing several operations in a row and not getting lost when indices change.

Let’s try a slightly longer “story” where you decide the steps, and then we’ll move it into code.

New list:

Python

nums = [10, 20, 30, 40, 50, 60]
Goal (in this order):

Change 30 to 35
Remove 10
Remove 60

First, just in words (no code yet):

What operation would you use for each step: update / remove / remove is obvious, but the important part is:

after step 1 and step 2, what does the list look like?

Try to write:

after step 1: [...]
after step 2: [...]

We’ll handle step 3 after that.

update index 2 to 35
pop the first item
remove the lat item
ater step 1: [10, 20. 35. 40, 50, 60]
after step 2: [20, 35, 40, 50, 60]
after step 3" [20, 35, 40, 50]

Your reasoning here is excellent, Ron 🎯

Let’s walk your steps and check each stage.

Start:

Python

nums = [10, 20, 30, 40, 50, 60]
0   1   2   3   4   5
1 ☐ Update index 2 to 35 ☑

Python

nums[2] = 35
[10, 20, 35, 40, 50, 60]
You wrote:

after step 1: [10, 20. 35. 40, 50, 60]

That’s correct (just some dots instead of commas in typing).

2️⃣ Pop the first item ☑

That means nums.pop(0):

Before pop:

Python

```
[10, 20, 35, 40, 50, 60]
#  0   1   2   3   4   5
```

After pop(0):

Python

```
[20, 35, 40, 50, 60]
#  0   1   2   3   4
```

You wrote:

after step 2: [20, 35, 40, 50, 60]

Also correct 👍

3️⃣ Remove the last item ☑

You said:

remove the lat item

after step 3" [20, 35, 40, 50]

That matches what nums.pop() would do:

Python

```
[20, 35, 40, 50, 60]  ->  [20, 35, 40, 50]
```

So your whole mental model is solid here.

Now I want to connect this to actual Python code in the editor.

Here’s the sequence as Python:

Python

```
nums = [10, 20, 30, 40, 50, 60]

nums[2] = 35      # step 1
nums.pop(0)       # step 2
nums.pop()        # step 3
print(nums)       # see the final result
```

Next step:

Open the code editor, type this exact code in, run it, and then hit Submit code.

After you submit, I'll check it and then we'll do one more, slightly trickier index-shift challenge together.

Python

```
nums = [10, 20, 30, 40, 50, 60]

nums[2] = 35      # step 1
nums.pop(0)       # step 2
nums.pop()        # step 3
print(nums)       # see the final result
```

Nice work running that, Ron — the output [20, 35, 40, 50] matches exactly what you predicted earlier 🎯

You've just:

updated by index
popped from the front
popped from the end
all in the correct order, and your index reasoning held up.
Now let's push the index-shift idea a bit harder with a small twist.

Start from:

Python

```
nums = [5, 10, 15, 20, 25, 30]
#         0   1   2   3   4   5
```

Goal, in this order:

Remove 10 using remove(10)
Then remove 25 using its index (using pop(index)), after step 1 has already happened

Step 1 is easy:

Python

```
nums.remove(10)
```

After that, the list is:

Python

```
[5, 15, 20, 25, 30]
# 0   1   2   3   4
```

Now the key question:

After step 1, what index should we pass to pop(...) to remove 25?

Just answer with the index number (like 3).

3

Exactly — 3 is right 🎯

After:

Python

```
nums = [5, 10, 15, 20, 25, 30]
nums.remove(10)
```

we get:

Python

```
[5, 15, 20, 25, 30]
# 0   1   2   3   4
25 is at index 3, so nums.pop(3) will remove it.
```

This is the core pattern of the lesson:

after every change, re-draw the list with indices, then choose the next index.

Want to turn this into real code once more?

If yes, here’s your task:

In the editor, start with:

```
nums = [5, 10, 15, 20, 25, 30]
```

Then:

remove 10 by value
remove 25 by index (the correct one after the first removal)
print(nums) at the end

Run it, then hit Submit code so I can check.

Python

```
nums = [5, 10, 15, 20, 25, 30]

nums.remove(10)
nums.pop(3)
print(nums)
```

That’s perfect, Ron ☑

Your code:

Python

```
nums = [5, 10, 15, 20, 25, 30]

nums.remove(10)
nums.pop(3)
print(nums)
```

Output:

Python

```
[5, 15, 20, 30]
```

Matches exactly what we expect:

After remove(10) → [5, 15, 20, 25, 30]
After pop(3) → removes 25 → [5, 15, 20, 30]

You’re clearly tracking how the indices change after each mutation 👏

Now I want to check one more thing from this lesson:

can you read a plain English requirement and turn it into the right sequence of operations and indices?

Here’s the scenario (just think it through, no code yet):

Python

letters = ["a", "b", "c", "d", "e", "f"]
Goal, in this order:

Change "c" to "Z"
Remove "a"
Remove "f"

Step by step, can you tell me:

After step 1, what does the list look like?
After step 2, what does the list look like?
After step 3, what does the list look like?

Write your three answers as three lists, like:

after 1: [...]
after 2: [...]
after 3: [...]
after 1: ['a', 'b', 'z', 'd', 'e', 'f']
after 2: ['b', 'z', 'd', 'e', 'f']
after : ['b', 'z', 'd', 'e']

Nice, Ron — your reasoning is spot on, just one tiny detail to fix 🔍

Start:

Python

```
letters = ["a", "b", "c", "d", "e", "f"]
#              0    1    2    3    4    5
```

1️⃣ Change "c" to "Z" (index 2):

Python

```
["a", "b", "Z", "d", "e", "f"]
```

You wrote 'z' instead of 'Z', but structurally it's correct.

2️⃣ Remove "a" (first element):

Python

```
["b", "Z", "d", "e", "f"]
#   0    1    2    3    4
```

Matches what you wrote 👍

3️⃣ Remove "f" (last element):

Python

```
["b", "Z", "d", "e"]
```

Also matches your answer.

So structurally, you nailed all three steps 🎯

Now I want to give you one small "which index is it now?" question to really lock in the index-shift idea.

Take this list and operations:

Python

```
nums = [100, 200, 300, 400, 500]

nums.pop(1)      # step 1
nums.pop()       # step 2
nums[1] = 250    # step 3
```

I'll show you a quick multiple-choice question about which value ends up at a certain index.

What do you think? Ready for that quick check?

sure

What do you think the answer is? Take your time with the question below 👇

Given this code:

```
nums = [100, 200, 300, 400, 500]

nums.pop(1)      # step 1
nums.pop()       # step 2
nums[1] = 250    # step 3
```

After all three steps, what is the value at index 1?

A.

200

B.

250

C.

300

D.

400

Nice job, Ron — 250 is exactly right 🎯

Step by step:

Python

```
nums = [100, 200, 300, 400, 500]
#           0    1    2    3    4
```

1. nums.pop(1) → remove 200:

Python

```
[100, 300, 400, 500]
# 0    1    2    3
```

2. nums.pop() → remove last (500):

Python

```
[100, 300, 400]
# 0    1    2
```

3. nums[1] = 250 → update index 1:

Python

```
[100, 250, 400]
# 0    1    2
```

So the value at index 1 is 250 ☑

You’re clearly tracking how the list changes after each mutation — that’s exactly the skill this lesson is about.

From what I’ve seen, you can:

Chain multiple operations (update, pop(index), pop(), remove(value)) in the right order
Recalculate indices after each change and reason about the final state

That covers the mastery goals for this lesson.

Do you feel ready to wrap up this lesson and move on to the next one, or do you want one more challenge of this type first?

I'm ready to move on

Excellent work today, Ron 🎉

Quick recap of what you just mastered:

You can chain list operations in Python:
list[index] = value (update)
pop(index) and pop() (remove by index / last element)
remove(value) (remove by value)
You’re careful to recalculate indices after every change, not using “stale” indices.
You can read a plain-English story (“change this, remove that”) and turn it into a clear sequence of index-based steps and code.

That’s exactly what “solving problems with arrays” at this level is about 👌

As you go forward, any time you’re editing lists, keep this mental loop:

Write the list with indices.
Do one operation.
Re-draw the list with new indices.
Then decide the next step.

You’re ready for the next lesson.

When you’re set, hit Next in the Learn tab, and I’ll keep going with you there 🚀